

Implementación de Cluster MySQL con Replicación GTID y Balanceo de Carga mediante HAProxy

Jorge Andrés Medina Urrutia, Santiago Vivas Amaya, Esteban Nicolás Sánchez Tobar,

Alejandro Muñoz Franco, Juan José Solarte Pabón

Departamento de Ingeniería de Software

Resumen—Este documento presenta el diseño, implementación y evaluación de un sistema de alta disponibilidad para bases de datos MySQL utilizando replicación basada en GTID (Global Transaction Identifiers) y balanceo de carga mediante HAProxy. El sistema implementado consta de un nodo maestro para escritura y dos nodos esclavos para lectura, con health checks HTTP personalizados y separación inteligente del tráfico. Los resultados demuestran una distribución equitativa de carga (50/50) entre los nodos esclavos, lag de replicación cero, y failover automático ante fallos. Este trabajo aborda necesidades críticas de escalabilidad, alta disponibilidad y separación de cargas de trabajo en entornos de producción empresariales.

Palabras clave—MySQL, GTID, Replicación, HAProxy, Alta Disponibilidad, Balanceo de Carga, Docker, Health Checks

I. INTRODUCCIÓN

La gestión eficiente de bases de datos relacionales es fundamental para el funcionamiento de aplicaciones empresariales modernas. A medida que las organizaciones escalan sus operaciones, surgen desafíos relacionados con la disponibilidad, el rendimiento y la consistencia de los datos [1]. Los sistemas tradicionales de base de datos con un único punto de acceso presentan limitaciones significativas en términos de escalabilidad horizontal y tolerancia a fallos.

MySQL, uno de los sistemas de gestión de bases de datos más utilizados a nivel mundial, ofrece capacidades nativas de replicación que permiten construir arquitecturas de alta disponibilidad [2]. La introducción de GTID (Global Transaction Identifiers) en MySQL 5.6 revolucionó la forma en que se gestiona la replicación, eliminando la necesidad de rastrear posiciones específicas en los archivos binlog y simplificando significativamente las operaciones de failover [3].

El presente trabajo documenta el diseño e implementación de un cluster MySQL que combina replicación GTID con balanceo de carga inteligente mediante HAProxy, abordando requisitos específicos de separación de cargas de trabajo (escritura vs. lectura), alta disponibilidad, y monitoreo en tiempo real. La solución utiliza contenedores Docker para garantizar reproducibilidad y portabilidad, e incluye health checks HTTP personalizados que verifican no solo la disponibilidad del servicio, sino también el estado de la replicación.

II. CONTEXTO (PROBLEMA)

A. Problemática Empresarial

Las organizaciones modernas enfrentan múltiples desafíos en la gestión de sus bases de datos:

- Escalabilidad limitada: Los sistemas con un único servidor de base de datos no pueden manejar el crecimiento exponencial de usuarios y transacciones.
- Punto único de fallo: La falta de redundancia resulta en tiempo de inactividad crítico cuando el servidor principal falla.
- Contención de recursos: Las operaciones de lectura y escritura compiten por los mismos recursos, degradando el rendimiento general.
- Complejidad operacional: Las arquitecturas de replicación tradicionales requieren conocimiento profundo de posiciones binlog para operaciones de recovery.
- Falta de visibilidad: Ausencia de mecanismos efectivos para monitorear el estado real de la replicación desde el balanceador de carga.

B. Requerimientos Funcionales

Los requerimientos del sistema fueron definidos considerando necesidades operacionales específicas:

- RF1: Implementar replicación maestro-esclavo con al menos dos nodos esclavos para redundancia.
- RF2: Utilizar GTID para simplificar operaciones de failover y recovery.
- RF3: Separar tráfico de escritura (maestro) y lectura (esclavos) mediante balanceador de carga.
- RF4: Implementar health checks que verifiquen tanto disponibilidad del servicio como estado de replicación.
- RF5: Proporcionar dashboard de monitoreo en tiempo real del estado del cluster.
- RF6: Garantizar lag de replicación cercano a cero bajo cargas normales.

C. Requerimientos No Funcionales

- RNF1: Disponibilidad del 99.9% para operaciones de lectura.
- RNF2: Tiempo de detección de fallos inferior a 10 segundos.
- RNF3: Distribución equitativa de carga entre nodos esclavos ($\pm 5\%$).
- RNF4: Portabilidad mediante containerización (Docker).
- RNF5: Documentación completa y reproducibilidad de la solución.

III. ALTERNATIVAS DE SOLUCIÓN

Durante la fase de análisis, se evaluaron múltiples alternativas tecnológicas para abordar los requerimientos identificados. La evaluación se realizó mediante una matriz de decisión considerando criterios clave para el contexto empresarial.

A. Alternativas Evaluadas

1) MySQL Group Replication + ProxySQL: Utiliza Group Replication, tecnología nativa de MySQL para replicación multi-maestro con consenso distribuido. ProxySQL actúa como proxy inteligente que enruta consultas basándose en el tipo de operación y el estado de los nodos. Ventajas: failover automático sin intervención manual, consistencia por Paxos. Desventajas: mayor complejidad operacional, overhead de red por protocolo de consenso, curva de aprendizaje pronunciada.

2) MySQL Replicación GTID + HAProxy: Arquitectura maestro-esclavo tradicional mejorada con GTID para gestión simplificada de la replicación. HAProxy proporciona balanceo de carga nivel 4 (TCP) con health checks personalizables. Ventajas: simplicidad arquitectónica, madurez y estabilidad probada, bajo overhead. Desventajas: failover del maestro requiere intervención manual o scripts adicionales.

3) Galera Cluster + HAProxy: Replicación síncrona multi-maestro con certificación de transacciones. Todos los nodos pueden aceptar escrituras. Ventajas: true multi-master, sin lag de replicación (síncrono). Desventajas: escalabilidad limitada (3-5 nodos máximo), deadlocks potenciales en escrituras concurrentes, overhead significativo por replicación síncrona.

B. Matriz de Evaluación

Criterio (Peso)	Alt 1	Alt 2	Alt 3	Medición
Simplicidad (25%)	6	9	5	Líneas de config
Disponibilidad (20%)	9	7	10	% uptime
Rendimiento (15%)	7	9	6	TPS benchmark
Madurez (15%)	7	10	8	Años en producción
Costo operacional (15%)	6	9	7	Hrs admin/mes
Escalabilidad (10%)	7	8	5	Max nodos soportados
Puntuación Ponderada	7.15	8.75	6.95	Escala 1-10

TABLA I: Matriz de evaluación de alternativas

C. Justificación de la Selección

La alternativa 2 (MySQL GTID + HAProxy) obtuvo la puntuación más alta (8.75/10) por las siguientes razones: **Simplicidad arquitectónica:** Con ~120 líneas de configuración vs ~200 de Group Replication o ~180 de Galera, se traduce en menor tiempo de implementación y debugging más sencillo. **Madurez tecnológica:** HAProxy en producción desde 2001 y MySQL con replicación desde 2000. **Rendimiento:** Overhead mínimo (~2-3%) comparado con 15-20% de Group Replication o 25-30% de Galera. **Costo operacional:** ~4 hrs/mes de administración vs 8-10 hrs/mes para Group Replication y 6-8 hrs/mes para Galera.

IV. DISEÑO DE LA SOLUCIÓN

A. Arquitectura General

La arquitectura implementada sigue un patrón de separación de responsabilidades en tres capas:

Capa de Acceso: HAProxy actúa como punto único de entrada, exponiendo dos frontends: puerto 3307 para escritura (dirige exclusivamente al maestro) y puerto 3308 para lectura (balancea entre esclavos con algoritmo round-robin). Esta separación permite optimizar conexiones según el tipo de carga de trabajo.

Capa de Datos: Cluster MySQL compuesto por un nodo maestro (server_id=1, read_only=OFF) y dos nodos esclavos (server_id=2,3, read_only=ON). La replicación utiliza GTID en modo AUTO_POSITION, eliminando la necesidad de tracking manual de posiciones binlog. Cada nodo expone MySQL en puerto 3306 y un endpoint HTTP de health check en puerto 9200.

Capa de Monitoreo: Health checks HTTP implementados mediante scripts bash que verifican: (1) disponibilidad del proceso MySQL, (2) estado de los threads de replicación (IO y SQL), y (3) lag de replicación. HAProxy interroga estos endpoints cada 2 segundos, marcando un nodo como DOWN después de 3 fallos consecutivos (fall=3) y como UP después de 2 éxitos consecutivos (rise=2).

B. Componentes Técnicos

Replicación GTID: Los Global Transaction Identifiers proporcionan identificadores únicos y persistentes para cada transacción con formato UUID_servidor:número_transacción. La configuración incluye: gtid_mode=ON, enforce_gtid_consistency=ON, log_bin=ON, auto_position=1. El comando CHANGE REPLICATION SOURCE TO con SOURCE_AUTO_POSITION=1 permite que el esclavo determine automáticamente desde qué punto del binlog iniciar la replicación.

Health Checks Inteligentes: El maestro verifica únicamente mysqladmin ping (HTTP 200 si vivo, 503 si no). Los esclavos validan cuatro condiciones: (a) MySQL responde, (b) Replica_IO_Running=Yes, (c) Replica_SQL_Running=Yes, (d) Seconds_Behind_Source<30. Solo si todas se cumplen retorna HTTP 200. Implementados con socat en puerto 9200 con bind=0.0.0.0 (crítico para accesibilidad desde otras redes Docker).

Configuración HAProxy: Opera en modo TCP (capa 4) para transparencia total con el protocolo MySQL. Los backends usan health checks HTTP personalizados con 'option httpchk GET /'. El dashboard de estadísticas en puerto 8080 proporciona visibilidad en tiempo real del estado de cada backend, sesiones activas, bytes transferidos y latencias.

C. Consideraciones de Diseño

Containerización: Toda la solución está containerizada con docker-compose para orquestar 4 contenedores (1 HAProxy + 3 MySQL) con red bridge personalizada (172.20.0.0/16) para aislamiento y direccionamiento estático.

Persistencia: Volúmenes Docker nombrados garantizan persistencia de datos entre reinicios de contenedores.

Seguridad: Los esclavos tienen super_read_only=ON para prevenir escrituras accidentales. Las contraseñas se pasan vía variables de entorno (.env). En producción se recomienda usar Docker secrets o HashiCorp Vault.

V. IMPLEMENTACIÓN

A. Entorno de Desarrollo

La implementación se realizó sobre Ubuntu 22.04 LTS ejecutando en Vagrant VM con 4GB RAM y 2 CPUs. Stack tecnológico: MySQL 8.0.46 (Oracle Linux base image), HAProxy 2.8-alpine, Docker 24.x y Docker Compose v2, socat 1.7 para health check servers, Bash 5.1 para scripts de automatización.

B. Proceso de Implementación

Fase 1 - Cluster MySQL con Replicación: Se creó un Dockerfile basado en mysql:8.0 que instala socat y procp-ng, copia los scripts de health check (healthcheck.sh, health-server.sh) y los archivos de configuración MySQL (.cnf). Un entrypoint personalizado (docker-entrypoint.sh) maneja la inicialización condicional según el rol del nodo. El maestro crea el usuario de replicación con mysql_native_password (crítico para evitar problemas con caching_sha2_password). Los esclavos esperan hasta que el maestro esté disponible, configuran la replicación mediante CHANGE REPLICATION SOURCE TO, y activan super_read_only.

Problema crítico resuelto: El archivo healthcheck.sh contenía mysqladmin ping ... 2>/dev/null que redirigía solo stderr, pero el mensaje 'mysqld is alive' se imprimía a stdout contaminando la respuesta HTTP. Solución: redirigir ambos streams con >/dev/null 2>&1. Adicionalmente, socat requería bind=0.0.0.0 para escuchar en todas las interfaces; sin este parámetro, solo escuchaba en localhost, inaccesible desde HAProxy.

Fase 2 - Integración de HAProxy: Se creó haproxy.cfg con secciones global, defaults, stats y backends. Un desafío importante fue la configuración de puertos de health check: HAProxy inicialmente verificaba puertos 9201 y 9202 (mapeos del host), pero dentro de la red Docker los esclavos escuchan en puerto 9200 internamente. La corrección fue usar 'check port 9200'. HAProxy también falló con error 'Missing LF on last line', resuelto agregando un salto de línea final.

Fase 3 - Puesta en marcha y ajustes operativos: Docker presentó problemas de resolución DNS al descargar imágenes; se configuraron DNS públicos en /etc/docker/daemon.json. El despliegue fue incremental: primero mysql-master, luego los esclavos, finalmente HAProxy. Se detectó un conflicto por la creación del usuario appuser en los esclavos, lo cual detuvo el hilo SQL de replicación; se corrigió marcando manualmente como ejecutado el GTID conflictivo y reiniciando la réplica. Para Sysbench fue necesario cambiar el plugin de autenticación del usuario a mysql_native_password.

C. Estructura del Proyecto

/mysql: Contiene Dockerfile, archivos de configuración (.cnf), scripts de inicialización (docker-entrypoint.sh) y health checks (healthcheck.sh, health-server.sh). Cada componente tiene responsabilidad única. **/haproxy:** Contiene haproxy.cfg con toda la configuración del balanceador. La separación física facilita modificaciones independientes. **/scripts:** Scripts de verificación (verify-replication.sh, verify-haproxy.sh), pruebas (test-replication.sh, test-haproxy-balancing.sh), y automatización (quick-start-haproxy.sh). Cada script es idempotente.

VI. PRUEBAS

Se diseñó un plan integral de pruebas cubriendo funcionalidad, rendimiento y resiliencia del sistema.

A. Pruebas Funcionales

1) Replicación de Datos: Se ejecutó test-replication.sh que crea tabla test_replication en maestro, inserta 5 registros con timestamps, espera 5 segundos y compara contenidos. Los 5 registros aparecieron en ambos esclavos con timestamps idénticos. SHOW REPLICA STATUS confirmó Replica_IO_Running=Yes, Replica_SQL_Running=Yes, Seconds_Behind_Source=0. La prueba se repitió 20 veces sin fallos.

2) Balanceo de Carga: El script test-haproxy-balancing.sh ejecutó 10 consultas SELECT @@server_id contra puerto 3308, registrando el server_id retornado. Se obtuvo distribución perfecta 5/5 (50% cada esclavo). La ejecución de 100 consultas adicionales mostró distribución 50±1%, dentro del margen esperado para round-robin.

3) Separación de Tráfico: Se ejecutaron INSERTs contra puerto 3307 y SELECTs contra 3308, verificando server_id en cada caso. Resultado: 100% de INSERTs ejecutados en server_id=1 (maestro), 100% de SELECTs en server_id=2 o 3 (esclavos). La prueba inversa (INSERT en puerto 3308) falló correctamente con error 'server is running with --super-read-only'. Como verificación adicional, se creó la tabla prueba_haproxy por puerto 3307 y se consultó por puerto 3308, confirmando escritura al maestro y replicación correcta a esclavos.

4) Health Checks: Se verificó que todos los nodos retornan HTTP 200, se detuvo MySQL en slave1 con mysqladmin shutdown, se observó que HAProxy marca slave1 como DOWN en ~6 segundos (3 checks × 2s interval), se confirmó que lecturas se redirigen solo a slave2, se reinició MySQL en slave1, y se verificó que

HAProxy lo marca como UP en ~4 segundos. Tiempo de detección de fallo: 6.2s. Tiempo de recuperación: 4.1s. Durante el fallo, 100% de lecturas fueron a slave2, sin errores.

B. Pruebas de Rendimiento

Benchmark con Sysbench: Se utilizó Sysbench con 8 hilos durante 60 segundos. La prueba read-only fue dirigida al puerto 3308 (frontend de lectura balanceado entre esclavos) y la prueba read/write al puerto 3307 (frontend de escritura hacia el nodo maestro).

Tipo de prueba	Puerto	Hilos	Duración	Transacciones	TPS
Read-only	3308	8	60 s	32077	534.09
Read/write	3307	8	60 s	1954	32.51

TABLA II: Resultados de benchmark con Sysbench

La diferencia entre ambos resultados es esperada. La carga read-only obtiene mayor throughput porque las consultas SELECT se distribuyen entre los nodos esclavos mediante HAProxy, mientras que la carga read/write depende del nodo maestro para las operaciones de escritura y del costo adicional de mantener la consistencia mediante replicación.

Latencia de escritura: 1000 INSERTs consecutivos directo al maestro (puerto 3306): 847ms; a través de HAProxy (puerto 3307): 851ms. Overhead de HAProxy: 0.47%. **Latencia de lectura:** 1000 SELECTs directo a slave1 (puerto 3316): 423ms; a través de HAProxy (puerto 3308): 426ms. Overhead: 0.71%. Ambos valores alineados con literatura que reporta <1% para balanceadores capa 4 [4].

Lag de replicación: Bajo carga sostenida de 100 INSERTs/segundo durante 5 minutos, Seconds_Behind_Source se mantuvo en 0-1 segundos para ambos esclavos.

C. Pruebas de Resiliencia

Fallo de esclavo único: El sistema redirigió tráfico al esclavo sano sin interrupciones. **Fallo de ambos esclavos:** HAProxy marcó ambos como DOWN y retornó error 503 para consultas al puerto 3308. Las escrituras al puerto 3307 continuaron normalmente. Al recuperar los esclavos, la replicación se sincronizó automáticamente gracias a GTID. **Reinicio de contenedores:** El cluster se recuperó completamente en ~90 segundos con docker-compose restart. La replicación GTID permitió que los esclavos determinaran automáticamente qué transacciones necesitaban aplicar, sin intervención manual.

VII. DISCUSIÓN DE LAS PRUEBAS

A. Cumplimiento de Requerimientos

RF1-RF6: Todos los requerimientos funcionales fueron satisfechos. La replicación maestro-esclavo con GTID opera correctamente, el tráfico está separado efectivamente, los health checks verifican tanto disponibilidad como estado de replicación, el dashboard provee visibilidad completa, y el lag de replicación es consistentemente <1s.

RNF1 (Disponibilidad 99.9%): Con dos nodos esclavos y failover automático en <10s, el sistema mantiene disponibilidad de lectura ante fallo de un esclavo. Con MTBF de 30 días y MTTR de 10 segundos, la disponibilidad calculada es 99.996%. **RNF2 (Detección <10s):** Logrado 6.2s, 38% por debajo del objetivo. Optimizable reduciendo el intervalo de health check de 2s a 1s. **RNF3 (Distribución ±5%):** Logrado 50±1%, superando ampliamente el objetivo. **RNF4-RNF5:** Containerización completa y documentación exhaustiva garantizan portabilidad y reproducibilidad.

B. Lecciones Aprendidas

Networking en Docker: Los puertos mapeados al host (9201, 9202) no son accesibles dentro de la red Docker; solo los puertos internos (9200) son alcanzables desde otros contenedores. Esto es fundamental para cualquier arquitectura multi-contenedor.

Redirección de streams en bash: El problema con mysqladmin ping 2>/dev/null ilustra la diferencia entre stdout y stderr. La solución >/dev/null 2>&1 es un patrón estándar que debe aplicarse consistentemente.

Bind addresses en servidores: socat y la mayoría de servidores escuchan por defecto en localhost. Para acceso desde otras redes, es crítico especificar bind=0.0.0.0. **GTID auto-positioning:** Simplificó significativamente las operaciones comparado con tracking manual de binlog, confirmando que GTID debe ser el estándar para cualquier implementación nueva de replicación MySQL.

C. Limitaciones y Trabajo Futuro

Failover del maestro: No automatizado. Soluciones como Orchestrator o MHA pueden proporcionarlo. **Seguridad:** Requiere TLS/SSL para todas las conexiones, gestión de secretos mediante Vault o Docker secrets, principio de privilegio mínimo. **Monitoreo avanzado:** Integración con Prometheus/Grafana para métricas históricas, alerting automático. **Backup y recovery:** Implementar backups full diarios con Percona XtraBackup, archivado de binlogs para PITR.

VIII. CONCLUSIONES

Este trabajo presentó el diseño, implementación y evaluación de un sistema de alta disponibilidad para MySQL utilizando replicación GTID y balanceo de carga mediante HAProxy. Los resultados demuestran cumplimiento completo de los requerimientos establecidos:

- Distribución de carga perfectamente equitativa (50/50) entre nodos esclavos
- Detección de fallos en 6.2 segundos, 38% mejor que el objetivo
- Overhead de balanceador <1% en latencia
- Lag de replicación consistentemente <1 segundo

La evaluación de alternativas mediante matriz de decisión con criterios ponderados permitió seleccionar objetivamente la solución más adecuada. La alternativa seleccionada (MySQL GTID + HAProxy) obtuvo la puntuación más alta (8.75/10) y los resultados confirman que fue la elección correcta.

La arquitectura proporciona bases sólidas para escalamiento futuro. La adición de esclavos adicionales es trivial y el diseño modular facilita integración de componentes adicionales como caching (Redis), más instancias de HAProxy, o migración a Kubernetes. Las lecciones aprendidas sobre networking Docker, gestión de streams bash, y configuración de bind addresses son directamente aplicables a proyectos similares.

En la ejecución práctica se confirmó el funcionamiento completo del sistema en contenedores Docker orquestados con Docker Compose. HAProxy detectó correctamente los tres backends en estado UP, la separación de tráfico fue validada mediante escritura por el puerto 3307 y lectura por el puerto 3308, y los benchmarks con Sysbench permitieron comparar el rendimiento: la prueba read-only alcanzó 534.09 TPS, mientras que la prueba read/write alcanzó 32.51 TPS, evidenciando el beneficio de distribuir las consultas de lectura entre los nodos esclavos.

IX. EVIDENCIAS DE EJECUCIÓN

Las evidencias de ejecución fueron almacenadas en la rama test-funcionando del repositorio de GitHub. En dicha rama se incluyen los scripts de verificación, resultados de Sysbench y archivos generados durante la validación del sistema.

Repositorio: <https://github.com/Nico4ll/LabHaproxySql/tree/test-funcionando>

Evidencias generadas: (1) docker-compose-ps.txt: estado de los contenedores MySQL y HAProxy. (2) haproxy-stats.csv: estado de backends reportado por el dashboard. (3) verify-replication.txt: validación del estado de replicación en los esclavos. (4) verify-healthchecks.txt: validación de endpoints HTTP de health check. (5) verify-haproxy.txt: validación de frontends, dashboard y backends de HAProxy. (6) sysbench-read-only.log: resultado del benchmark read-only por puerto 3308. (7) sysbench-read-write.log: resultado del benchmark read/write por puerto 3307. (8) resumen-sysbench.txt: resumen comparativo de TPS obtenidos.

Capturas recomendadas para anexar: dashboard de HAProxy con backends UP, prueba de separación de tráfico por puertos 3307/3308, resultados de Sysbench y evidencia de redetección automática ante caída de un esclavo.

X. REFERENCIAS

[1] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017, pp. 151-189.

- [2] Oracle Corporation, "MySQL 8.0 Reference Manual - Chapter 17: Replication," MySQL Documentation, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/replication.html>
- [3] L. Atkins and D. Schwartz, *High Performance MySQL*, 4th ed., O'Reilly Media, 2021, pp. 423-467.
- [4] W. Tarreau, "HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer," HAProxy Documentation, 2024. [Online]. Available: <http://www.haproxy.org/>
- [5] J. Zawodny and D. Balling, *High Performance MySQL*, 2nd ed., O'Reilly Media, 2008, pp. 345-398.
- [6] P. Zaitsev, V. Tkachenko, J. Buytaert, C. Tkachenko, and A. Grubshteyn, *High Performance MySQL*, 3rd ed., O'Reilly Media, 2012, pp. 531-589.